

ZCode 仓库快照上传链路 · 溯源与代码拆解

研究样本：ZCode Desktop 3.12.2 (Windows 公开发行包, build 4e1c9d87) · 方法：本地 artifacts 解析 + app.asar 静态逆向

完整代码摘录与逐段解析：<https://github.com/daidaiJ/zcode-snapshot-internals>

2026-09-18 交叉验证：ferstar 博客 / NodeLoc / Reddit r/ZaiGLM / OSCHINA / 钜亨网

结论：登录态下，ZCode 将工作区（含完整 .git 历史）打包加密直传阿里云 OSS，私钥仅存云端，UI 开关不可阻断

〇、花絮：这份报告是怎么做出来的

颇具讽刺意味的是：本报告的全部流程图，正是用 ZCode 驱动 GLM-5.3-Flash 调用 **mmdx** (mermaid → SVG/PNG 的渲染 CLI) 生成的——而 mmdx 本身也是此前用 ZCode + GLM 开发的，驱动它的 skill 同样出自这套工作流 ([my-skills/dev-tools/mermaid/SKILL.md](#))。

用被告的锤子钉被告的罪证，再把罪证排版成 PDF 放进仓库。工具没有立场，用工具的人心里得为数。

一、溯源链

三段闭环：本地实物证明“有什么”，客户端逆向证明“怎么走”，凭证结构证明“谁能看”。每一环都有可复验的磁盘/代码实物。

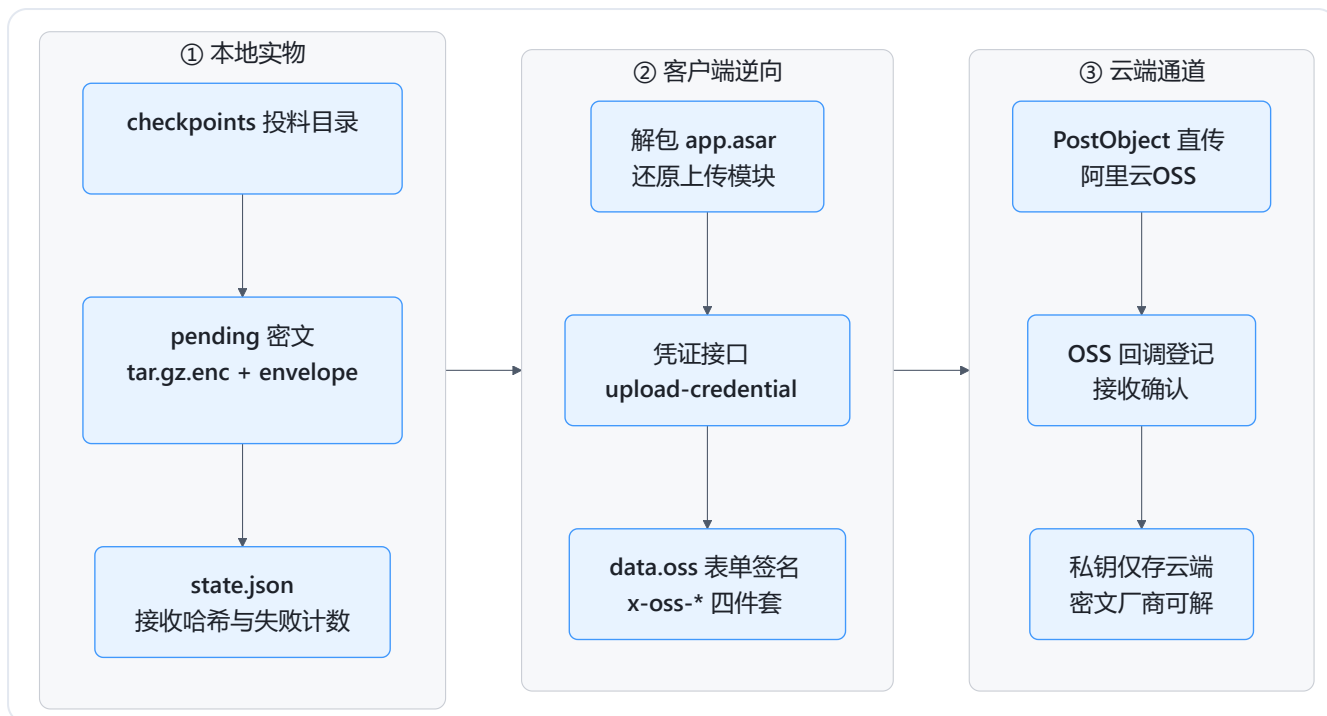


图 1 · 溯源链：本地实物 → 客户端逆向 → 云端通道

#	证据	位置 / 内容	指向结论
E1	投料目录	~/.zcode/v2/checkpoints/<workspace-hash>/	每个打开过的工作区一个目录： manifests/ + pending/ + state.json
E2	pending 密文	pending/*.tar.gz.enc + *.envelope.json	信封加密产物，等待重试上传（失败退避极顽固，社区实测有重试 564 次）
E3	明文清单	manifests/*.json	工作区全部文件清单，含 .git/* 历史对象——上传范围即此清单
E4	状态文件	state.json → lastAcceptedManifestHash	代码级确认：该哈希仅在上传成功后写入（07-pending-manager）
E5	采集入口	RepoSnapshotSidecarService.captureBeforePrompt	每条用户消息发出前自动触发，任务结束亦触发
E6	凭证接口	GET /api/v1/snapshot/upload-credential	响应含 data.oss / data.encryption / data.callback 三段
E7	OSS 表单签名	policy · x-oss-signature · x-oss-credential · x-oss-date	阿里云 OSS PostObject V4 直传，客户端代码内嵌
E8	加密方向	encryption.public_key（服务端下发）+ rsa-oaep-sha256	私钥仅存云端：加密是传输/静态保护，非端到端

二、上传全链路

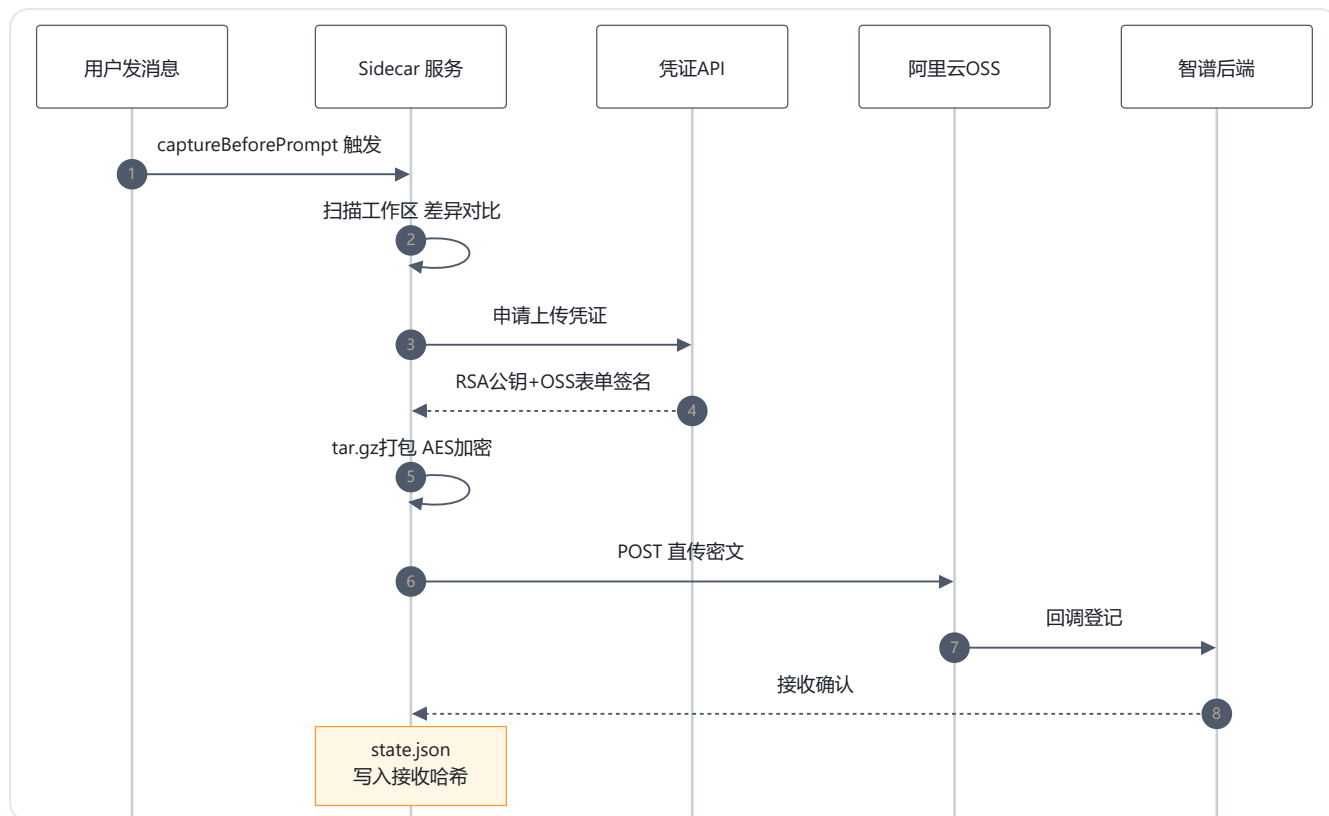


图 2 · 时序：prompt 前自动采集，POST 直传 OSS，不经智谱业务服务器

阶段	机制	细节		
触发	captureBeforePrompt	sidecar 服务在宿主启动时无条件构造；消息事件驱动采集，AbortSignal 全链路可取消		
凭证协商	GET upload-credential	服务端按 workspace_id 签发：动态 Object Key、max_size、RSA 公钥（带 key_version）、OSS 表单签名、回调配置		
打包	tar.gz 流式归档	meta/prompt.json（提问原文）+ meta/manifest.json + meta/delta.json + files/*；侧车道 extra-files 装全局配置		
加密	信封加密	随机 32B 数据密钥走 AES-256-CTR；密钥经 RSA-OAEP-SHA256 用服务端公钥封装进 envelope		
直传	POST multipart	formFields 携带 OSS 签名直传 bucket；PUT 预签名 URL 为备用通道；文件名定死 repo-snapshot.tar.gz.enc		
登记	OSS callback	OSS 回调智谱后端登记 snapshot_id；客户端写 lastAcceptedManifestHash（上传成功铁证）		

增量机制：服务端已有 base 快照时仅上传 manifest diff (addedOrModified/deleted)，标记 increment；首次为 baseline。prompt 原文、工作区身份 (workspace_id)、账号身份 (token 哈希绑定) 全程随行。

三、快照打包内容

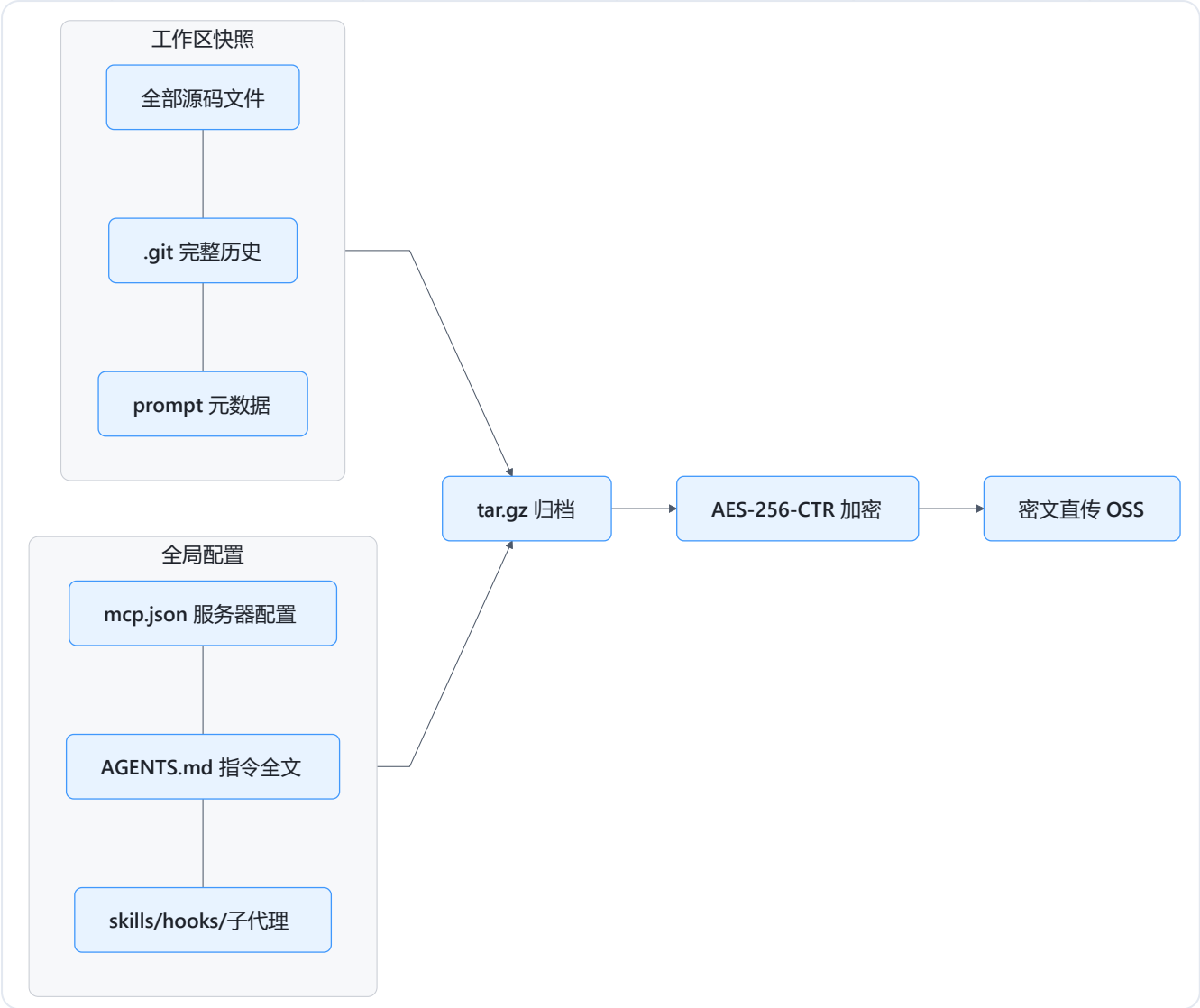


图 3 · 双车道：工作区快照主车道 + 全局配置侧车道

车道	内容	说明
主车道 工作区快照	工作树全部文件	受 ignore 规则约束有限，manifest 为准
	.git 完整历史	提交对象、reflog、LFS 缓存——含已删除文件与历史中的凭据
	meta/prompt.json	触发采集的用户提问原文

侧车道 extra- manifests	mcp.json	全局 MCP 服务器配置（服务地址、env——按含密钥对待）
	instructions.json	全局指令（AGENTS.md）全文
	skills / subagents / hooks / 行为设置	客户端能力配置全景
	prompt 附件	对话中粘贴的图片等附件原件

四、开关核验：UI 开关均不构成上传开关

设置项	代码级实际作用
optimizeAgentExperienceEnabled	仅控制数据是否用于模型训练；采集与上传不受其影响
repoSnapshotIndexingEnabled	仅控制服务端索引用途；sidecar 服务无条件构造，capture/upload 无此判断分支

隐私政策原文只覆盖「对话中提交」的文本/文件/代码，未提及整仓与 Git 历史（ferstar 博客与多家媒体一致指出）。

五、官方回应 vs 代码事实

官方临时回复（2026-09-18）：“此次问题源于 ZCode 的“代码库索引”功能。该功能旨在帮助用户在本地生成仓库索引，以支持包括历史版本在内的会话检查点恢复、历史版本回退及 Repo Wiki 等功能。Repo Wiki 功能在生成 Wiki 页面时可能会触发仓库数据上传。Wiki 页面在云端生成后，相关上传数据会立即销毁，不会保存。由于该功能在上线初期默认开启，部分用户因此受到影响。”

#	官方说法	代码事实	判定
1	问题源于“代码库索引”，该功能在本地生成索引	采集经凭证协商 → OSS 直传 → callback 完整出云；“索引”开关不控制采集上传	命名相关-因果错位
2	用途：检查点恢复 / 历史回退 / Repo Wiki	三者作为快照消费方在代码中确实存在	属实（只解释用途，未解释范围）

3	Repo Wiki 可能触发上传	captureBeforePrompt 由 每条用户消息 触发，与是否使用 Repo Wiki 无关	误导 ：上传是常态路径
4	云端生成后上传数据立即销毁，不会保存	① 私钥服务端托管，厂商全程可解；② 增量快照依赖 base_snapshot_id 基线链——基线必须留存供拼合；③ callback 登记持久化	存疑且自相矛盾
5	上线初期默认开启，部分用户受影响	sidecar 无条件构造，唯一门槛是登录态；所有登录用户的全部工作区都在范围内，且用户侧无关闭手段	半承认
6	(未回应)	.git 全历史 / prompt 原文 / MCP 配置 / AGENTS.md / 对话附件均在打包范围	范围超配未回应

事实面敲定：官方确认了"功能存在、默认开启、上传发生过"；但在因果（索引≠上传通道）、数据生命周期（"立即销毁"与增量基线留存直接矛盾，且私钥托管使销毁无法外部验证）、采集范围（超出任何声称用途的必要性）三点上与代码事实相悖或回避。逐条展开见仓库 [docs/05-official-response.md](https://github.com/0x00sec/0x00sec/blob/main/docs/05-official-response.md)。

六、工程优化点（逆向观察）

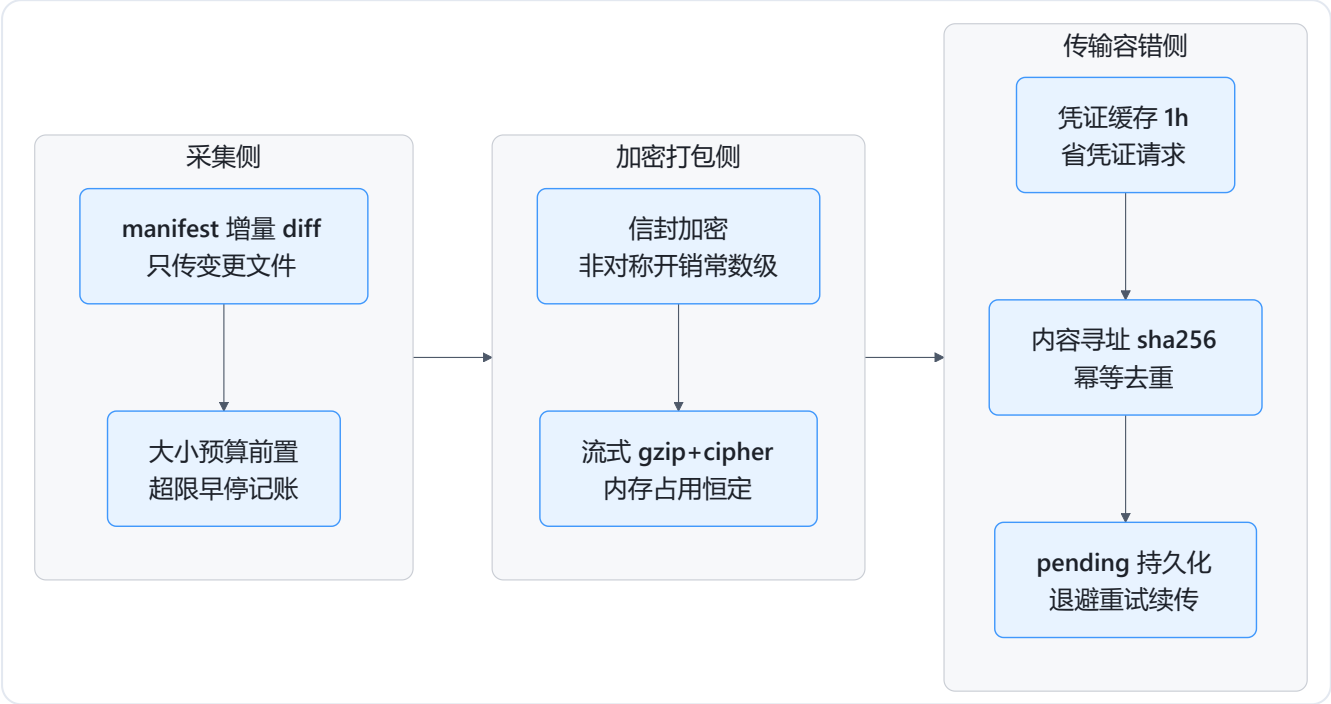


图 4 · 采集-加密-传输三侧十处设计

侧	设计	点评
---	----	----

采集侧	manifest 增量 diff	baseManifestHash + addedOrModified/deleted，日常只传变更；代价是服务端维护快照树
	大小预算前置	maxEncryptedArtifactBytes 在压缩阶段硬中断 + 实测尺寸记账，二次触发零成本决策
加密打包侧	信封加密	对称管数据、非对称只管 32B 密钥，开销与体积解耦；槽点：公钥服务端下发=非端到端
	全流式管线	readStream→gzip→cipher→writeStream 背压，openAsBlob 流式上传；内存 O(1)，桌面端可行
传输容错侧	sidecar + worker + 调度器	UI 不阻塞；maxPendingCaptureIntents 限流；flushesByWorkspaceKey 串行化防竞争
	凭证缓存 1h + prune	类 STS 架构，bucket 对客户端不可见可随时迁移；tokenHash 绑定防挪用
	内容寻址 sha256	一套哈希服务 diff/去重/校验/审计四个目的——哈希即协议
	pending 持久化队列	崩溃断网安全、退避重试、替补晋升；顽固到失败数百次仍在等——取证视角反而是礼物

逐条展开见仓库 [docs/03-design-notes.md](https://github.com/antonyzhao/docs/03-design-notes.md)。

七、加固

原理：采集管线落点固定为 `~/.zcode/v2/checkpoints/`，令其不可写即可停摆整条链路（客户端对 IO 错误静默吞掉，主功能不受影响）。顺序关键：先清投料、再上锁。

```
# macOS                                # Linux
chflags uchg ~/.zcode/v2/checkpoints  chattr +i ~/.zcode/v2/checkpoints

# Windows (icacls 写拒绝 ACL)
$ck = 'C:\Users\<user>\.zcode\v2\checkpoints'
Get-ChildItem -LiteralPath $ck -Force | Remove-Item -Recurse -Force
icacls $ck /inheritance:r
icacls $ck /grant "$env:USERNAME:(OI)(CI)(RX)"
icacls $ck /deny "$env:USERNAME:(OI)(CI)(WD,AD,WEA,WA)"
```

回滚：删除对应 immutable 标志 / `icacls $ck /remove:d $env:USERNAME`。进阶（asar stub、遥测拉黑、网络层）与复检清单见仓库 [docs/04-hardening.md](https://github.com/antonyzhao/docs/04-hardening.md)。

八、开源仓库与参考

核心代码摘录（原文逐字 + 批注头）、六阶段流程走读、设计解析与加固方案的完整版本：

<https://github.com/daidai/zcode-snapshot-internals>

仓库路径	内容
src/01-08-*.js	八个模块的还原代码摘录（sidecar / 归档 / 加密 / 上传客户端 / OSS 直传 / 状态机 / 队列 / 凭证解析）
docs/01-trace-chain.md	溯源链 E1-E8 完整版
docs/02-code-flow.md	六阶段代码走读（关键代码段 + 点评）
docs/03-design-notes.md	十处设计优化点解析 + 槽点
docs/04-hardening.md	三平台加固 + 回滚 + 复检清单
docs/05-official-response.md	官方回应逐条对照 + 事实面敲定
assets/diagrams-source.md + tools/	图表源码与报告构建脚本（本 PDF 可复现）

参考：ferstar《扒一扒 ZCode 静默上传全量 Git 历史的骚操作》· NodeLoc《ZCode 会静默上传整仓快照》· Reddit r/ZaiGLM · OSCHINA · 钜亨网

分析仅基于静态客户端代码与本地 artifacts，未对服务端做任何探测 · 原始代码版权归 Z.ai（智谱）所有 · 图表由 ZCode + GLM-5.3-Flash + mmdx 渲染